

Socket.IO Realtime System

Overview

Socket.IO powers:

- online/offline presence
- chat room membership
- chat laugh reaction events
- livestream rooms, comments, reactions, guest seat actions, and viewer counts
- notification fan-out through user-specific rooms

Current implementation

- Socket.IO is created in `server.js`
- each user joins a room keyed by their user id after `userConnected`
- online presence is tracked in in-memory `Map` objects
- livestream rooms use two room names for backward compatibility:
 - `livestream\_<streamId>`
 - `live:<streamId>`

Important socket events

Presence

- `userConnected`
- `userOnline`
- `userOffline`
- `requestOnlineUsers`
- outbound: `getOnlineUsers`, `userStatusChanged`

Chat

- `joinChatRoom`
- `leaveChatRoom`
- `chat\_laugh\_reaction`

Livestream

- `live\_host\_ready`
- `live\_host\_heartbeat`
- `live\_join`
- `live\_leave`
- `live\_comment`
- `live\_reaction`
- `live\_request\_guest\_seat`
- `live\_approve\_guest\_seat`
- `live\_decline\_guest\_seat`
- `live\_mute\_guest`
- `live\_kick\_guest`
- outbound: `live\_room\_joined`, `live\_viewer\_count`, `live\_ended`, `live\_started`, guest seat events

Event flow

```mermaid

flowchart TD

```
Client --> Socket[Socket.IO Event]
Socket --> InMemory[In-memory Maps / Sets]
Socket --> Mongo[MongoDB Updates]
Socket --> Rooms[User Rooms / Live Rooms]
Rooms --> Clients[Other Connected Clients]
```

...

## ## Known issues

- presence, dedupe, and room participant state are stored in process memory
- no Redis adapter or distributed room state exists
- reconnect semantics are partially handled with timers rather than a shared store

## ## Scaling concerns

- multi-instance deployment will fragment presence and livestream membership
- process restarts lose in-memory dedupe windows and presence state
- high fan-out live events still depend on a single socket gateway process

## ## Recommended improvements

1. add Redis adapter for Socket.IO
2. move presence and livestream participant state to Redis
3. formalize event schemas and versioning
4. capture metrics for room size, emit latency, and reconnect churn